Distributed Systems

Homework 3

Name: Hala Mohammad Barakat

This homework builds a fault-tolerant distributed banking transaction in Java. An Order Service (the client) asks a Banking Service (the server) over HTTP to debit an amount from an account. The connection can break, or the server can crash, before the request arrives, after it arrives but before the debit, or after the debit but before the confirmation reaches the client. The goal is **effectively-once** execution: a debit is never lost and never applied twice.

The project was built with Spec-Driven Development (SDD) using GitHub Spec Kit and the coding agent Claude Code. The specification is the single source of truth: every class and every test names the requirement it implements. The full project, with its whole history, is at https://github.com/HalaBarakat1/banking-sdd.

The required work is:

1. **At-least-once delivery:** the client retries with timeouts and exponential backoff when it gets no answer.
2. **Artificial idempotency:** the client generates a strictly increasing sequence key for each new transaction and sends it with an is_retry flag.
3. **Atomicity and persistence:** the server checks the account's last_sequence_key and debits in one atomic transaction; a repeated key is acknowledged without a second debit.
4. **Fault injection simulator:** request lost, server crash before execution, response lost after execution, and response slower than the client's timeout.
5. **SDD workflow:** constitution, specification, plan, tasks, then implementation and verification, each visible in the Git history.

1. The Spec-Driven Development workflow

The work followed the five Spec Kit phases in order. No code was written before the documents of the earlier phases were reviewed and committed.

1.1 The project constitution (constitution.md)

The constitution fixes the rules every later phase must follow, in six principles:

- **The spec is the single source of truth:** every requirement has an ID (FR, NFR, EC, SC, FS), and every task, class and test cites the IDs it serves.
- **Effectively-once money movement:** the key check and the balance change happen in one atomic transaction; decisions come only from stored state, never from is_retry; the answer is sent only after the commit.
- **Thread safety through the database:** SQLite's write lock is the single place where concurrent requests are serialized.
- **Bounded failure handling:** a timeout on every network call, bounded retries with exponential backoff and jitter, and UNKNOWN (never "failed") when retries run out.
- **Test-first and fault-proven:** a failing test is committed before the code that makes it pass.
- **Simplicity and observability:** only the JDK HTTP server and client, SQLite, Jackson and JUnit 5; one log line per request.

The constitution was amended twice (v1.0.1 and v1.1.0), each time in its own commit, when a later phase showed that a rule had to change.

1.2 The specification (spec.md)

The specification describes what the system must do and why, without any code: 5 user stories, 31 functional requirements, 5 non-functional requirements, 12 edge cases, 7 success criteria and the 5 fault scenarios FS-1 to FS-4b. The most important design decisions were made while writing it:

- **One record per decided key:** keeping only the last key and its outcome would give a late copy of an older key a different answer from the original, so every decided key gets its own transaction record.
- **Insufficient funds consumes the key:** "one key = one decision", so a delayed copy of a refused debit can never execute later.
- **Stale keys:** a key below the last key that was never decided is rejected as STALE_KEY and can never execute.
- **Gaps are allowed:** a client that crashes after reserving a key must not block its account forever.
- **Cancel burns a key:** if a request is stuck in the network and the client gives up, a cancel guarantees that the stuck copy can never execute.

1.3 The plan and the research (plan.md, research.md)

The plan turns the specification into an architecture. research.md records twelve technical decisions (R1 to R12), each with its reason and the alternatives that were rejected. data-model.md holds the tables and the decision table, the contracts describe the HTTP endpoints and the command line, and a traceability table links each requirement to its classes and tests. The key decisions are:

- **One BEGIN IMMEDIATE transaction per decision (R2):** the write lock is taken before the first read, which removes the race between checking a key and writing it.
- **WAL with synchronous=FULL (R3):** a committed decision survives a crash of the process.
- **Retry by status class (R4):** 2xx and 4xx are final; no answer, a timeout or any 5xx is retried with the same key.
- **Two fault hooks (R5):** one before the decision and one after the commit, so FS-3 and FS-4b act on a decision that is already saved.
- **A client journal in SQLite (R6):** reserving a key and recording the transaction happen in one step.

Planning also found contradictions in the specification, and they were fixed in the specification first. FR-011 called a late debit on a cancelled key a conflict, against User Story 5. FR-026 (resend every unresolved transaction on start) would resend the very debit the user starts the client to cancel. Reviewing the first fix of that problem showed it was flawed, and it was replaced: the switch to cancel is saved in the journal before anything is sent.

1.4 The tasks and the consistency checks (tasks.md, /speckit-analyze)

tasks.md breaks the plan into 58 test-first tasks grouped by user story. Before any code was written, /speckit-analyze checked the specification, the plan and the tasks against each other. The first run found 30 issues, 9 of them critical; for example, Jackson's default setting would have turned "amount": 2.5 into a debit of 2, and retries had no total deadline. A second run confirmed the fixes and found 9 smaller issues. All of them were fixed in the documents before task T001.

1.5 Implementation and verification

Every task was done test-first: the failing test was committed as test(Txxx), then the code that makes it pass as feat(Txxx), with the whole suite green. Tests that check behaviour an earlier task had already delivered are verification tests; before committing each one, it was shown to fail against a deliberate break of the code it guards.

2. The server: Banking Service

The server is a JDK HttpServer with 32 worker threads in front of a SQLite database (bank.db). It has two tables: accounts, which holds the balance and the last_sequence_key of each account, and transaction_records, which holds one row per decided key with the primary key (account_id, sequence_key).

2.1 The decision table: debit()

```java
public Answer debit(String accountId, DebitRequest request) {
    long key = request.sequenceKey();
    long amount = request.amount();
    return store.inImmediateTransaction(tx -> {
        // Row 1: unknown account. Not a decision: nothing is written (FR-012).
        Optional<AccountRow> found = tx.account(accountId);
        if (found.isEmpty()) {
            return answer(accountId, key, Outcome.UNKNOWN_ACCOUNT, false, null, null, "no account " + accountId);
        }
        AccountRow account = found.get();

        Optional<RecordRow> decided = tx.record(accountId, key);
        if (decided.isPresent()) {
            RecordRow record = decided.get();
            // Row 2: same key, different amount (FR-011). The original decision stands.
            if (record.operation() == Operation.DEBIT && record.amount() != amount) {
                return answer(accountId, key, Outcome.KEY_CONFLICT, false, null, null,
                        "key " + key + " was decided for amount " + record.amount());
            }
            // Row 3: replay the recorded decision (FR-008).
            return replay(record);
        }

        // Row 4: undecided and not above the last key (FR-009).
        if (key <= account.lastSequenceKey()) {
            return answer(accountId, key, Outcome.STALE_KEY, false, null, null,
                    "key " + key + " is not above the last key " + account.lastSequenceKey());
        }

        // Rows 5 and 6: a new key. Either way the key is consumed (FR-006).
        boolean enough = account.balance() >= amount;
        long balanceAfter = enough ? account.balance() - amount : account.balance();
        Outcome outcome = enough ? Outcome.EXECUTED : Outcome.INSUFFICIENT_FUNDS;
        tx.updateAccount(accountId, balanceAfter, key);
        tx.insertRecord(new RecordRow(accountId, key, Operation.DEBIT, amount, outcome, balanceAfter,
                Instant.now().toString()));
        return answer(accountId, key, outcome, false, amount, balanceAfter, message(outcome));
    });
}
```

This method is the core of the project. It implements the decision table of the data model inside one database transaction:

1. **Unknown account:** answers UNKNOWN_ACCOUNT and writes nothing.
2. **Same key, different amount:** answers KEY_CONFLICT; the original decision stands.

3. **Key already decided:** replays the recorded decision with duplicate=true and moves no money. This is what makes every retry safe.
4. **Stale key:** an undecided key that is not above the last key answers STALE_KEY and can never execute.
5. **New key:** executes the debit if the balance covers it, otherwise records INSUFFICIENT_FUNDS. Either way the key becomes the last key and gets its record.

The method never reads is_retry. The decision comes only from stored state, so a first-ever key marked as a retry still executes, and a copy marked as new is still a duplicate.

2.2 The atomic transaction: inImmediateTransaction()

```
public <T> T inImmediateTransaction(TransactionWork<T> work) {
    try (Connection c = open(); Statement s = c.createStatement()) {
        s.execute("BEGIN IMMEDIATE");
        T result;
        try {
            result = work.run(new LedgerTransaction(c));
        } catch (SQLException | RuntimeException | Error e) {
            rollback(s, e);
            throw e;
        }
        s.execute("COMMIT");
        return result;
    } catch (SQLException e) {
        throw new StoreException("transaction failed: " + e.getMessage(), e);
    }
}
```

Every decision runs between BEGIN IMMEDIATE and COMMIT on its own connection. IMMEDIATE takes the database write lock before the first read, so no other request can check the same key between this request's read and its write. If the

work throws, everything is rolled back. The result is returned only after COMMIT succeeds, so an answer is never sent for a decision that was not saved.

A test made this choice visible: when BEGIN IMMEDIATE was deliberately replaced with BEGIN DEFERRED, concurrent transactions first read and then collided on the write lock, SQLite refused one of them at once, and debits, even on different accounts, received 503.

2.3 Cancelling a key: cancel()

```
public Answer cancel(String accountId, CancelRequest request) {
    long key = request.sequenceKey();
    return store.inImmediateTransaction(tx -> {
        Optional<AccountRow> found = tx.account(accountId);
        if (found.isEmpty()) {
            return answer(accountId, key, Outcome.UNKNOWN_ACCOUNT, false, null, null, "no account " + accountId);
        }
        AccountRow account = found.get();

        Optional<RecordRow> decided = tx.record(accountId, key);
        if (decided.isPresent()) {
            RecordRow record = decided.get();
            return switch (record.outcome()) {
                case EXECUTED -> answer(accountId, key, Outcome.ALREADY_EXECUTED, true, record.amount(),
                        record.balanceAfter(), "too late: the debit was already executed");
                case INSUFFICIENT_FUNDS -> answer(accountId, key, Outcome.ALREADY_REJECTED, true, record.amount(),
                        record.balanceAfter(), "the debit was refused; nothing will be debited");
                case CANCELLED -> answer(accountId, key, Outcome.CANCELLED, true, null, null, "key was cancelled");
                default -> throw new IllegalStateException("record with outcome " + record.outcome());
            };
        }

        if (key <= account.lastSequenceKey()) {
            return answer(accountId, key, Outcome.STALE_KEY, false, null, null,
                    "key " + key + " is not above the last key " + account.lastSequenceKey()
                            + "; it never executed and never will");
        }

        tx.updateAccount(accountId, account.balance(), key);
        tx.insertRecord(new RecordRow(accountId, key, Operation.CANCEL, null, Outcome.CANCELLED,
                account.balance(), Instant.now().toString()));
        return answer(accountId, key, Outcome.CANCELLED, false, null, null, "key cancelled");
    });
}
```

cancel() follows the same pattern. A new key is burnt: a CANCEL record is written with no amount, the key becomes the last key and the balance does not change, so any later copy of that debit replays CANCELLED. If the key was already decided,

the answer is a replay: ALREADY_EXECUTED (the cancel came too late, with the original amount), ALREADY_REJECTED, or CANCELLED again.

2.4 The fault hooks in OperationHandler

```
Optional<FaultRule> fault = faults.choose(accountId, operation);
if (fault.isPresent() && fault.get().scenario().beforeDecision()) {
    FaultRule rule = fault.get();
    switch (rule.scenario()) {
        case DROP_REQUEST -> {
            dropped(accountId, request, rule, started);
            return;
        }
        case CRASH_BEFORE_EXECUTE -> {
            dropped(accountId, request, rule, started);
            crash.accept(Duration.ofMillis(rule.downtimeMs()));
            return;
        }
        case DELAY_BEFORE_EXECUTE -> {
            if (!pause(rule)) {
                return;
            }
        }
        default -> throw new IllegalStateException("not a before-decision scenario: " + rule.scenario());
    }
}

Answer answer;
int status;
try {
    answer = request.decision().get();
    status = StatusMapper.statusFor(operation, answer.outcome());
} catch (StoreException e) {
    boolean busy = e.isBusy();
    if (!busy) {
        LOG.log(Level.WARNING, "database failure for " + accountId, e);
    }
    answer = unavailable(accountId, request, busy ? "database busy, retry" : "database failure, retry");
    status = busy ? 503 : 500;
} catch (RuntimeException e) {
    LOG.log(Level.WARNING, "unexpected failure for " + accountId, e);
    answer = unavailable(accountId, request, "unexpected server error, retry");
    status = 500;
}

FaultRule after = fault.filter(r -> !r.scenario().beforeDecision()).orElse(null);
if (after != null && after.scenario() == FaultScenario.DROP_RESPONSE) {
    // FS-3: the decision is committed; close the connection without an answer.
    log(answer, operation, request, after, started);
    return;
}
if (after != null && after.scenario() == FaultScenario.DELAY_AFTER_EXECUTE && !pause(after)) {
    return;
}
respond(exchange, status, answer, operation, request, fault.orElse(null), started);
```

After the request is parsed and validated, the injector chooses at most one fault scenario for it. Before the decision, FS-1 closes the connection without deciding or answering, FS-2 does the same and asks the server to crash and restart, and FS-4a waits. After the commit, FS-3 closes the connection without answering and FS-4b waits before answering. The decision code never sees the injector, and every request still writes exactly one log line.

2.5 Choosing a fault: choose()

```
public Optional<FaultRule> choose(String accountId, Operation operation) {
    Armed a = armed.get();
    List<FaultRule> rules = a.config().rules();
    boolean anyProbability = false;
    for (int i = 0; i < rules.size(); i++) {
        FaultRule r = rules.get(i);
        if (!r.matches(accountId, operation)) {
            continue;
        }
        if (r.trigger() == FaultTrigger.NEXT_N) {
            if (a.remaining().get(i).getAndUpdate(n -> n > 0 ? n - 1 : 0) > 0) {
                return Optional.of(r);
            }
        } else {
            anyProbability = true;
        }
    }
    if (!anyProbability) {
        return Optional.empty();
    }
    double u = new SplittableRandom(a.seed() + a.draws().getAndIncrement()).nextDouble();
    double upper = 0;
    for (FaultRule r : rules) {
        if (r.trigger() == FaultTrigger.PROBABILITY && r.matches(accountId, operation)) {
            upper += r.probability();
            if (u < upper) {
                return Optional.of(r);
            }
        }
    }
    return Optional.empty();
}
```

NEXT_N rules are checked first, in order; the first one with count left fires, and its counter is an AtomicInteger, so it fires exactly count times even under many threads. Otherwise one number is drawn and the PROBABILITY rules are laid side by side, so each fires with exactly its own probability. The draw uses new SplittableRandom(seed + n) with n from an AtomicLong, so there is no shared mutable Random and a run with a fixed seed is reproducible.

2.6 Crash and restart: crashAndRestart()

```
public void crashAndRestart(Duration downtime) {
    if (!restarting.compareAndSet(false, true)) {
        return;
    }
    Thread restart = new Thread(() -> {
        try {
            Running old;
            synchronized (lifecycle) {
                if (closed) {
                    return;
                }
                old = running;
            }
            LOG.info("FS-2 crash: server stopped, restarting in " + downtime.toMillis() + " ms");
            stopRunning(old);
            Thread.sleep(downtime.toMillis());
            synchronized (lifecycle) {
                if (closed) {
                    return;
                }
                running = startRunning(port);
                restarts.incrementAndGet();
            }
            LOG.info("FS-2 crash: server restarted on port " + port);
        } catch (IOException e) {
            LOG.log(Level.SEVERE, "FS-2 crash: restart failed", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            restarting.set(false);
        }
    }, "banking-crash-restart");
    restart.setDaemon(true);
    restart.start();
}
```

For FS-2 the server stops listening at once, discards every in-memory object (store, handlers, worker threads), waits the configured downtime and starts again on the same database file and port. Everything the server knows about money afterwards comes from the database, exactly as after a real crash. The restart runs on its own thread and only once at a time, and the fault rules are kept so they keep firing.

3. The client: Order Service

The client keeps its own SQLite file (client.db) with a key counter and a journal of transactions, so it survives its own crash.

3.1 Reserving a key: reserve()

```java
public Optional<JournalEntry> reserve(String accountId, Operation operation, long amount) {
    if (!AccountIds.isValid(accountId)) {
        throw new IllegalArgumentException("account_id must be 1-64 chars of [A-Za-z0-9_-]: " + accountId);
    }
    if (amount <= 0) {
        throw new IllegalArgumentException("amount must be positive: " + amount);
    }
    return inTransaction(c -> {
        try (PreparedStatement s = c.prepareStatement(
                "SELECT 1 FROM journal WHERE account_id = ? AND state <> 'RESOLVED'")) {
            s.setString(1, accountId);
            try (ResultSet r = s.executeQuery()) {
                if (r.next()) {
                    return Optional.empty();
                }
            }
        }
        long key;
        try (Statement s = c.createStatement()) {
            s.executeUpdate("UPDATE key_counter SET last_key = last_key + 1 WHERE id = 1");
            try (ResultSet r = s.executeQuery("SELECT last_key FROM key_counter WHERE id = 1")) {
                r.next();
                key = r.getLong(1);
            }
        }
        String now = Instant.now().toString();
        try (PreparedStatement s = c.prepareStatement("INSERT INTO journal (" + COLUMNS + ") "
                + "VALUES (?, ?, ?, ?, 'PENDING', NULL, NULL, 0, ?, ?)")) {
            s.setLong(1, key);
            s.setString(2, accountId);
            s.setString(3, operation.name());
            s.setLong(4, amount);
            s.setString(5, now);
            s.setString(6, now);
            s.executeUpdate();
        }
        return Optional.of(new JournalEntry(key, accountId, operation, amount, JournalState.PENDING, null, null,
                0, now, now));
    });
}
```

In one transaction, reserve() checks that the account has no unresolved transaction, increments the key counter and inserts the journal entry as PENDING. A crash can therefore never leave a reserved key without its journal entry, and a key is never handed out twice. If the account is still blocked by an unresolved transaction, nothing is written and the call is refused.

3.2 The retry loop: send()

```
private TransactionResult send(long key) {
    JournalEntry entry = journal.find(key).orElseThrow();
    if (entry.state() == JournalState.RESOLVED) {
        return new TransactionResult(resultFor(entry.operation(), entry.outcome()), entry.accountId(), key,
                entry.outcome(), true, entry.balanceAfter(), 0, "already resolved");
    }
    RetryRun run = new RetryRun(policy, clock);
    while (run.mayAttempt()) {
        boolean isRetry = journal.countAttempt(key) > 0;
        AttemptResult result = entry.operation() == Operation.DEBIT
                ? http.debit(entry.accountId(), key, entry.amount(), isRetry, run.attemptTimeout())
                : http.cancel(entry.accountId(), key, isRetry, run.attemptTimeout());
        run.attempted();
        ClientLog.attempt(entry.operation(), entry.accountId(), key, run.attempts(), isRetry, result);
        if (result instanceof AttemptResult.Final done) {
            hooks.at(ClientHooks.Point.AFTER_RESPONSE_BEFORE_RECORD, entry);
            return finish(entry, done, run.attempts());
        }
        Duration wait = backoff.delayAfter(run.attempts());
        if (!run.mayAttempt() || !run.waitFits(wait)) {
            break;
        }
        try {
            sleeper.sleep(wait);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
    journal.markUnknown(key);
    ClientLog.note("op=" + entry.operation() + " account=" + entry.accountId() + " key=" + key
            + " result=unknown attempts=" + run.attempts());
    return new TransactionResult(Result.UNKNOWN, entry.accountId(), key, null, null, null, run.attempts(),
            "no final answer after " + run.attempts() + " attempts; the key is kept for a later resend");
}
```

send() performs one send run and works as follows:

1. **Read the entry from the journal:** the run always sends the entry's current operation (debit or cancel); an entry that is already resolved is reported without sending.
2. **Count the attempt first:** countAttempt() is committed before sending, and is_retry is true exactly when an earlier attempt was counted.

3. **Send with the same key and amount:** the run's time limit shortens the last attempt's timeout.
4. **Final answer:** a 2xx or 4xx ends the run, and the outcome is recorded.
5. **No answer:** a timeout, a lost connection or a 5xx waits for the backoff and tries again, while attempts and time remain; otherwise the entry becomes UNKNOWN and keeps its key.

3.3 Exponential backoff with jitter: delayAfter()

```
public Duration delayAfter(int attemptsMade) {
    if (attemptsMade < 1) {
        throw new IllegalArgumentException("attemptsMade must be >= 1: " + attemptsMade);
    }
    long base = policy.baseBackoff().toNanos();
    long max = policy.maxBackoff().toNanos();
    int doublings = attemptsMade - 1;
    // Stop doubling once the cap is reached, so large attempt numbers cannot overflow.
    long d = base;
    for (int i = 0; i < doublings && d < max; i++) {
        d = d > max / 2 ? max : d * 2;
    }
    d = Math.min(d, max);
    long half = d / 2;
    return Duration.ofNanos(half + (long) (random.getAsDouble() * (d - half)));
}
```

After n attempts the wait is d/2 plus a random part of d/2, with d = min(maxBackoff, base × 2^(n−1)). The wait grows exponentially, is never zero, and spreads out the retries of many clients. Doubling stops at the cap, so large attempt numbers cannot overflow. With the defaults, even the worst case ends within 30 seconds.

3.4 Owning a reserved key: sendOwned()

```
private TransactionResult sendOwned(long key) {
    while (!active.add(key)) {
        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            JournalEntry entry = journal.find(key).orElseThrow();
            return new TransactionResult(Result.UNKNOWN, entry.accountId(), key, null, null, null, 0,
                    "interrupted while another thread was sending the key; it is kept for a later resend");
        }
    }
    try {
        return send(key);
    } finally {
        active.remove(key);
    }
}
```

The soak test found a client-side double payment here. A recovery running in one thread could send a key that a debit in another thread had just reserved; the debit then answered REFUSED, and a caller retrying after REFUSED paid twice. The server was correct (each key executed once), so only a realistic, busy client revealed it. Now the debit that reserved a key waits for any other sender and reports the key's real outcome.

3.5 Cancelling from the client: cancel() and switchToCancel()

```java
public TransactionResult cancel(String accountId, long key) {
    if (!active.add(key)) {
        return refusedCancel(accountId, key, "key " + key + " is being sent right now");
    }
    try {
        Optional<JournalEntry> current = journal.find(key);
        if (current.isPresent() && current.get().operation() == Operation.CANCEL
                && current.get().state() != JournalState.RESOLVED && current.get().accountId().equals(accountId)) {
            return send(key); // already switched by an earlier run: just send the cancel
        }
        Optional<JournalEntry> switched = journal.switchToCancel(accountId, key);
        if (switched.isEmpty()) {
            return refusedCancel(accountId, key, "key " + key + " is not an unresolved debit of " + accountId);
        }
        hooks.at(ClientHooks.Point.AFTER_CANCEL_SWITCH, switched.get());
        return send(key);
    } finally {
        active.remove(key);
    }
}
```

```java
public Optional<JournalEntry> switchToCancel(String accountId, long key) {
    boolean switched = inTransaction(c -> {
        try (PreparedStatement s = c.prepareStatement("UPDATE journal SET operation = 'CANCEL', attempts = 0, "
                + "state = 'PENDING', updated_at = ? WHERE sequence_key = ? AND account_id = ? "
                + "AND operation = 'DEBIT' AND state <> 'RESOLVED'")) {
            s.setString(1, Instant.now().toString());
            s.setLong(2, key);
            s.setString(3, accountId);
            return s.executeUpdate() == 1;
        }
    });
    return switched ? find(key) : Optional.empty();
}
```

The client first commits the switch from debit to cancel in the journal and only then sends the cancel. If the client crashes in between, the next start sends the cancel, never the old debit. A cancel of a key this client is sending right now is refused locally.

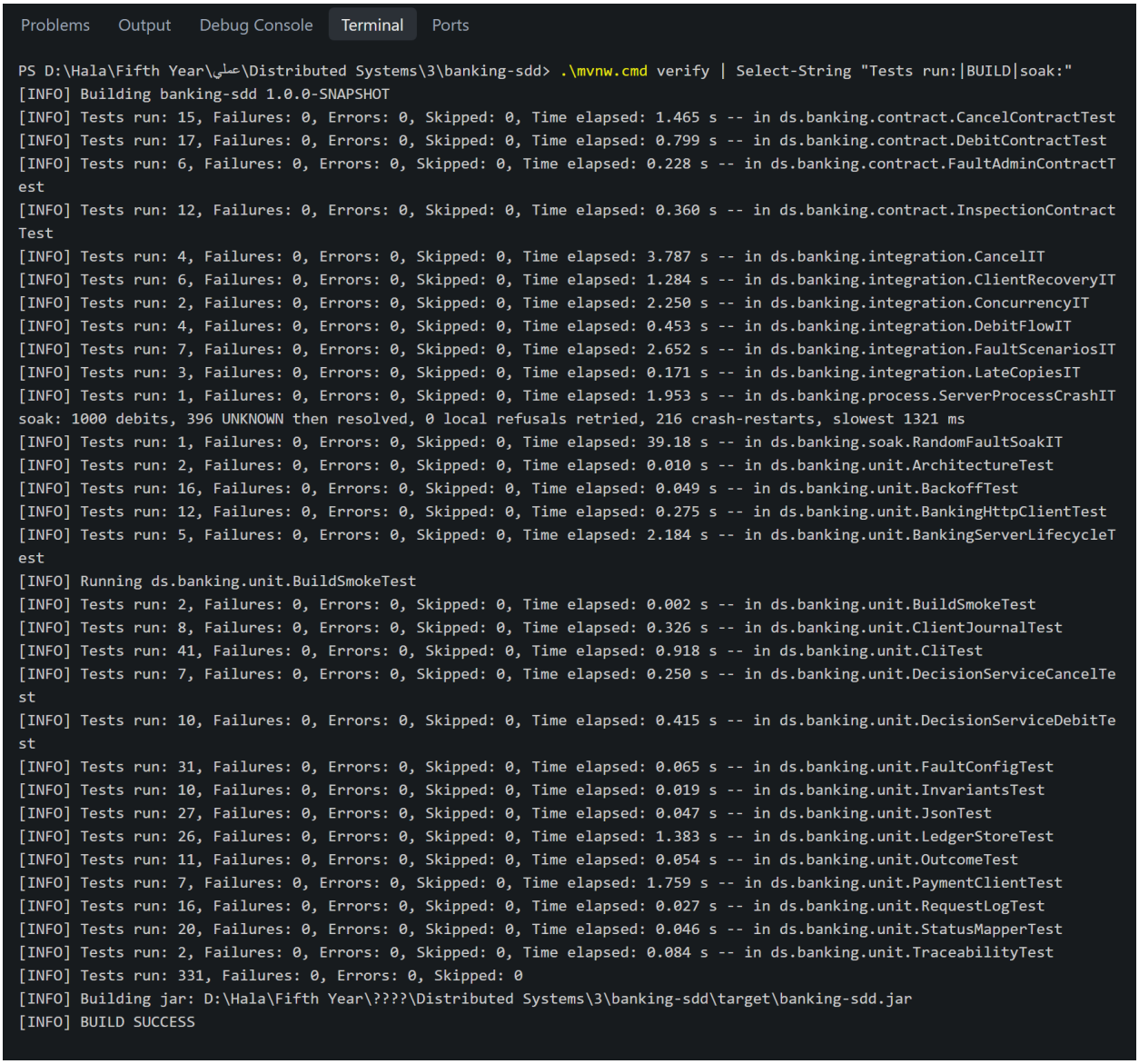
3.6 Strict JSON: buildMapper()

```
private static ObjectMapper buildMapper() {
    JsonMapper mapper = JsonMapper.builder()
            .propertyNamingStrategy(PropertyNamingStrategies.SNAKE_CASE)
            .enable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
            .enable(DeserializationFeature.FAIL_ON_NULL_FOR_PRIMITIVES)
            .enable(DeserializationFeature.FAIL_ON_TRAILING_TOKENS)
            .disable(DeserializationFeature.ACCEPT_FLOAT_AS_INT)
            .disable(MapperFeature.ALLOW_COERCION_OF_SCALARS)
            .build();
    mapper.coercionConfigDefaults()
            .setCoercion(CoercionInputShape.String, CoercionAction.Fail)
            .setCoercion(CoercionInputShape.Float, CoercionAction.Fail)
            .setCoercion(CoercionInputShape.EmptyString, CoercionAction.Fail);
    mapper.coercionConfigFor(LogicalType.Boolean)
            .setCoercion(CoercionInputShape.Integer, CoercionAction.Fail);
    return mapper;
}
```

Jackson's lenient defaults are switched off: by default it would turn "amount": 2.5 into 2, accept "250" as a number and default a missing is_retry to false. Every such body is now INVALID_REQUEST, and nothing is rounded or defaulted. The consistency check found this before any code existed.

4. Testing

4.1 Running all the tests



mvnw.cmd verify runs all 331 tests: 253 unit, 50 contract, 26 integration, 1 process and 1 soak test, and then builds the jar. Every test's display name starts with the requirement IDs it proves, and TraceabilityTest fails the build if a requirement in spec.md is cited by no test. The terminal pictures in this section show the real output of the commands.

4.2 Starting the server

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target\banking-sdd.jar server --port 8080 --db data\b
ank.db --seed A=1000
seed A=1000: CREATED
Banking Service listening on http://127.0.0.1:8080 (database D:\Hala\Fifth Year\????\Distributed Systems\3\banking-sdd\data
\bank.db)
2026-11-03T21:35:22.647Z INFO op=DEBIT account=A key=1 is_retry=false decision=EXECUTED outcome=EXECUTED fault=DROP_RESPONSE
latency_ms=19
2026-11-03T21:35:22.737Z INFO op=DEBIT account=A key=1 is_retry=true decision=DUPLICATE outcome=EXECUTED fault=none latency_
ms=4
2026-11-03T21:35:24.115Z INFO op=DEBIT account=A key=2 is_retry=false decision=DROPPED outcome=- fault=DROP_REQUEST latency_
ms=1
2026-11-03T21:35:24.214Z INFO op=DEBIT account=A key=2 is_retry=true decision=EXECUTED outcome=EXECUTED fault=none latency_m
s=7
2026-11-03T21:35:25.552Z INFO op=DEBIT account=A key=3 is_retry=false decision=DROPPED outcome=- fault=CRASH_BEFORE_EXECUTE
latency_ms=1
Nov 04, 2026 12:35:25 AM ds.banking.server.BankingServer lambda$crashAndRestart$2
INFO: FS-2 crash: server stopped, restarting in 1000 ms
Nov 04, 2026 12:35:26 AM ds.banking.server.BankingServer lambda$crashAndRestart$2
INFO: FS-2 crash: server restarted on port 8080
2026-11-03T21:35:27.121Z INFO op=DEBIT account=A key=3 is_retry=true decision=EXECUTED outcome=EXECUTED fault=none latency_m
s=6
2026-11-03T21:35:30.548Z INFO op=DEBIT account=A key=4 is_retry=true decision=EXECUTED outcome=EXECUTED fault=none latency_m
s=11
2026-11-03T21:35:31.451Z INFO op=DEBIT account=A key=4 is_retry=false decision=DUPLICATE outcome=EXECUTED fault=DELAY_BEFORE
_EXECUTE latency_ms=3007
2026-11-03T21:35:34.031Z INFO op=DEBIT account=A key=5 is_retry=true decision=DUPLICATE outcome=EXECUTED fault=none latency_
ms=9
2026-11-03T21:35:34.918Z INFO op=DEBIT account=A key=5 is_retry=false decision=EXECUTED outcome=EXECUTED fault=DELAY_AFTER_E
XECUTE latency_ms=3012
2026-11-03T21:35:41.367Z INFO op=CANCEL account=A key=6 is_retry=false decision=EXECUTED outcome=CANCELLED fault=none latenc
y_ms=11
2026-11-03T21:35:43.027Z INFO op=DEBIT account=A key=6 is_retry=false decision=DUPLICATE outcome=CANCELLED fault=DELAY_BEFOR
E_EXECUTE latency_ms=4009
2026-11-03T21:35:43.602Z INFO op=DEBIT account=A key=6 is_retry=true decision=DUPLICATE outcome=CANCELLED fault=DELAY_BEFORE
_EXECUTE latency_ms=4010
```

The server seeds account A with 1000 and listens on port 8080. Its log shows one line per request during the run below: the account, the key, is_retry, the decision, the outcome, the injected fault and the time taken. The two FS-2 messages in the middle show the server stopping and restarting on the same port.

4.3 FS-3: the response is lost

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\علمي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"DROP_RESPONSE","trigger":"NEXT_N","count":1}]}'
{"rules":[{"scenario":"DROP_RESPONSE","trigger":"NEXT_N","count":1}]}
PS D:\Hala\Fifth Year\علمي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db deb
it A 100
2026-11-03T21:35:22.668Z INFO op=DEBIT account=A key=1 attempt=1 is_retry=false result=retryable reason=no response: java.i
o.IOException: HTTP/1.1 header parser received no bytes
2026-11-03T21:35:22.800Z INFO op=DEBIT account=A key=1 attempt=2 is_retry=true result=final status=200 outcome=EXECUTED dupl
icate=true
SUCCEEDED: debit executed
{"result":"SUCCEEDED","account_id":"A","sequence_key":1,"outcome":"EXECUTED","duplicate":true,"balance_after":900,"attempt
s":2}
PS D:\Hala\Fifth Year\علمي\Distributed Systems\3\banking-sdd> curl.exe -s http://127.0.0.1:8080/accounts/A
{"account_id":"A","initial_balance":1000,"balance":900,"last_sequence_key":1}
```

The debit executes and is saved, but the response is not delivered. The client retries with the same key and is_retry=true, and the server replays the decision with duplicate=true: the balance drops from 1000 to 900 once.

4.4 FS-1: the request is lost

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\علمي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"DROP_REQUEST","trigger":"NEXT_N","count":1}]}'
{"rules":[{"scenario":"DROP_REQUEST","trigger":"NEXT_N","count":1}]}
PS D:\Hala\Fifth Year\علمي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db deb
it A 100
2026-11-03T21:35:24.137Z INFO op=DEBIT account=A key=2 attempt=1 is_retry=false result=retryable reason=no response: java.i
o.IOException: HTTP/1.1 header parser received no bytes
2026-11-03T21:35:24.278Z INFO op=DEBIT account=A key=2 attempt=2 is_retry=true result=final status=200 outcome=EXECUTED dupl
icate=false
SUCCEEDED: debit executed
{"result":"SUCCEEDED","account_id":"A","sequence_key":2,"outcome":"EXECUTED","duplicate":false,"balance_after":800,"attempt
s":2}
```

The server closes the connection without deciding (DROPPED). The retry is the first request to reach the decision, so it executes with duplicate=false.

4.5 FS-2: the server crashes before executing

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"CRASH_BEFORE_EXECUTE","trigger":"NEXT_N","count":1}]}'
{"rules":[{"scenario":"CRASH_BEFORE_EXECUTE","trigger":"NEXT_N","count":1,"downtime_ms":1000}]}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db deb
it A 100
2026-11-03T21:35:25.572Z INFO op=DEBIT account=A key=3 attempt=1 is_retry=false result=retryable reason=no response: java.i
o.IOException: HTTP/1.1 header parser received no bytes
2026-11-03T21:35:25.689Z INFO op=DEBIT account=A key=3 attempt=2 is_retry=true result=retryable reason=no response: java.ne
t.ConnectException
2026-11-03T21:35:25.887Z INFO op=DEBIT account=A key=3 attempt=3 is_retry=true result=retryable reason=no response: java.ne
t.ConnectException
2026-11-03T21:35:26.298Z INFO op=DEBIT account=A key=3 attempt=4 is_retry=true result=retryable reason=no response: java.ne
t.ConnectException
2026-11-03T21:35:27.180Z INFO op=DEBIT account=A key=3 attempt=5 is_retry=true result=final status=200 outcome=EXECUTED dupl
icate=false
SUCCEEDED: debit executed
{"result":"SUCCEEDED","account_id":"A","sequence_key":3,"outcome":"EXECUTED","duplicate":false,"balance_after":700,"attempt
s":5}
```

After the first attempt the server is down for one second, so attempts 2 to 4 get ConnectException. The client keeps waiting longer between attempts, and attempt 5 reaches the restarted server, which executes the debit once.

4.6 FS-4a and FS-4b: the response is slower than the client's timeout

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"DELAY_BEFORE_EXECUTE","trigger":"NEXT_N","count":1,"delay_ms":3000}]}'
{"rules":[{"scenario":"DELAY_BEFORE_EXECUTE","trigger":"NEXT_N","count":1,"delay_ms":3000}]}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db deb
it A 100
2026-11-03T21:35:30.447Z INFO op=DEBIT account=A key=4 attempt=1 is_retry=false result=retryable reason=timed out: request t
imed out
2026-11-03T21:35:30.624Z INFO op=DEBIT account=A key=4 attempt=2 is_retry=true result=final status=200 outcome=EXECUTED dupl
icate=false
SUCCEEDED: debit executed
{"result":"SUCCEEDED","account_id":"A","sequence_key":4,"outcome":"EXECUTED","duplicate":false,"balance_after":600,"attempt
s":2}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"DELAY_AFTER_EXECUTE","trigger":"NEXT_N","count":1,"delay_ms":3000}]}'
{"rules":[{"scenario":"DELAY_AFTER_EXECUTE","trigger":"NEXT_N","count":1,"delay_ms":3000}]}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db deb
it A 100
2026-11-03T21:35:33.894Z INFO op=DEBIT account=A key=5 attempt=1 is_retry=false result=retryable reason=timed out: request t
imed out
2026-11-03T21:35:34.098Z INFO op=DEBIT account=A key=5 attempt=2 is_retry=true result=final status=200 outcome=EXECUTED dupl
icate=true
SUCCEEDED: debit executed
{"result":"SUCCEEDED","account_id":"A","sequence_key":5,"outcome":"EXECUTED","duplicate":true,"balance_after":500,"attempt
s":2}
```

In FS-4a the server waits 3 seconds before deciding; the client times out after 2 seconds and retries, the retry executes, and the delayed first copy finishes later as a duplicate. In FS-4b the server decides first and then waits; the retry receives the saved decision with duplicate=true.

4.7 The state after five faults

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -s -X DELETE http://127.0.0.1:8080/admin/faults
{"rules":[]}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -s http://127.0.0.1:8080/accounts/A
{"account_id":"A","initial_balance":1000,"balance":500,"last_sequence_key":5}
```

Five debits of 100 under five different faults leave the balance at exactly 500 with last key 5: every debit executed exactly once.

4.8 Cancel: a debit stuck at the server

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -X PUT http://127.0.0.1:8080/admin/faults -H "Content-
Type: application/json" -d '{"rules":[{"scenario":"DELAY_BEFORE_EXECUTE","trigger":"NEXT_N","count":2,"operation":"DEBIT","d
elay_ms":4000}]}'
{"rules":[{"scenario":"DELAY_BEFORE_EXECUTE","trigger":"NEXT_N","count":2,"operation":"DEBIT","delay_ms":4000}]}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db --a
ttempts 2 --timeout-ms 500 --backoff-ms 50 debit A 100
2026-11-03T21:35:39.515Z INFO op=DEBIT account=A key=6 attempt=1 is_retry=false result=retryable reason=timed out: request t
imed out
2026-11-03T21:35:40.094Z INFO op=DEBIT account=A key=6 attempt=2 is_retry=true result=retryable reason=timed out: request ti
med out
2026-11-03T21:35:40.109Z INFO op=DEBIT account=A key=6 result=unknown attempts=2
UNKNOWN: no final answer after 2 attempts; the key is kept for a later resend
{"result":"UNKNOWN","account_id":"A","sequence_key":6,"attempts":2}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db can
cel A 6
2026-11-03T21:35:41.444Z INFO op=CANCEL account=A key=6 attempt=1 is_retry=false result=final status=200 outcome=CANCELLED d
uplicate=false
CANCELLED: key cancelled
{"result":"CANCELLED","account_id":"A","sequence_key":6,"outcome":"CANCELLED","duplicate":false,"attempts":1}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> curl.exe -s http://127.0.0.1:8080/accounts/A
{"account_id":"A","initial_balance":1000,"balance":500,"last_sequence_key":6}
PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> java -jar target/banking-sdd.jar client --db data/client.db jou
rnal A
{"sequence_key":6,"account_id":"A","operation":"CANCEL","amount":100,"state":"RESOLVED","outcome":"CANCELLED","attempts":
1,"created_at":"2026-11-03T21:35:38.843897300Z","updated_at":"2026-11-03T21:35:41.452545600Z"}
```

The server holds the debit for 4 seconds, so the client gives up with UNKNOWN (exit code 3). The cancel then burns key 6. When the two delayed copies of the debit wake up, both replay CANCELLED (in the server log above), and the balance stays at 500.

4.9 Deliberate breaks

Each verification test was shown to fail when the code it guards was deliberately broken, then the break was reverted:

- **BEGIN DEFERRED instead of BEGIN IMMEDIATE:** ConcurrencyIT failed because concurrent debits received 503.
- **The stale-key check removed:** LateCopiesIT failed because an old key executed (200 instead of 409).

- **State kept in a file per process:** ServerProcessCrashIT, which kills the real server process with destroyForcibly(), failed with a real double debit after the restart.
- **The record lookup skipped:** the soak test failed with "client reports S9#5 not executed, the server executed it".
- **A requirement with no test:** TraceabilityTest reported the temporary ID FR-099 as cited by no test.

4.10 The soak test

RandomFaultSoakIT sends 1,000 debits over 10 accounts in parallel while all five scenarios fire at random, 10% each. In the run above, 396 debits first ended UNKNOWN and were then resolved, and the server crashed and restarted 216 times. Afterwards every balance was exact, no key was executed twice, and the client's journal agreed with the server's records.

5. The development history

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Hala\Fifth Year\عملي\Distributed Systems\3\banking-sdd> git log --oneline --reverse | Select-Object -First 24
cf6abd3 chore: scaffold GitHub Spec Kit for Claude Code
3b6d618 docs(constitution): ratify v1.0.0
1bfbed7 docs(spec): specify effectively-once distributed debit
4e9569e docs(plan): design architecture for effectively-once debit
b19c307 docs(constitution): amend to v1.0.1, tests live in tests/java
14e5478 docs(spec): amend FR-003, FR-011 and FR-026 after planning
be46a1e docs(spec): save cancel intention before recovery (FR-026, FR-027)
7a4128c docs(constitution): amend to v1.1.0 after /speckit-analyze
d3b8385 docs(spec): amend after /speckit-analyze
ad766ec docs(plan): align design with analyze findings
36266ee docs(tasks): break the plan into 58 test-first tasks
aa45ef9 feat(T001): add Maven Wrapper 3.3.2 pinned to Maven 3.9.9
43b59ad feat(T002): add pom.xml with the constitution's stack
e7c4d5f feat(T003): ignore build output and runtime databases, fix line endings
2c5107a test(T004): add failing smoke test for the jar entry point
578d406 feat(T004): add jar entry point with usage and invalid-argument exit code
cfdd962 docs(plan): close NULL hole in the transaction_records CHECK
7e87e05 test(T005): add failing strict JSON contract test
acb5bcb test(T006): add failing outcome retry-class test
b750c75 feat(T012): add protocol records, outcomes and strict JSON
a7dff58 test(T007): add failing status-mapping test
249d930 feat(T013): map (operation, outcome) to HTTP status
5094405 test(T008): add failing SQLite store test
96fc54f feat(T014): add SQLite ledger store with IMMEDIATE transactions
```

The 78 commits show the phases in order: the Spec Kit scaffold, the constitution, the specification, the plan, the amendments that planning caused, the tasks, and then the implementation in test → feat pairs. Mistakes were fixed in visible fix commits, never by rewriting history.

6. Engineering reflection

6.1 Design gaps found before any code existed

- Keeping only the last key would have given late copies of older keys a different answer; this led to one record per key.
- A stuck request could have executed after the client gave up; this led to the cancel operation instead of a deadline, which would depend on clocks agreeing.

- Two contradictions in the specification, and a flawed first fix, were caught during planning.
- The consistency check caught 2.5 becoming 2, the missing retry deadline and a retry counter that could block an UNKNOWN transaction forever.

6.2 Bugs the tests found during implementation

- **A NULL hole in a database CHECK:** NULL > 0 is unknown, so a debit with no amount would have been accepted.
- **A lost log line in FS-4:** the line was written after sending, which fails when the client has already left.
- **The client double payment:** found by the soak test and fixed with sendOwned().
- **A flaky test setup:** log-capturing tests could lose their handler because loggers are only weakly held.

6.3 Directing the coding agent

I acted as the system architect: I reviewed every phase before it was committed, made the design decisions (consuming the key on insufficient funds, the stale-key lookup, gaps, the client journal and the cancel operation), and questioned details that led to better designs. The agent's mistakes, for example a file garbled by a PowerShell encoding problem and a miscounted number in a commit message, were corrected in visible commits.

6.4 Limitations

- One server instance at a time, with no replication.

- Process crashes are tested for real; power loss relies on synchronous=FULL and is not tested.
- No deposits, refunds, authentication or TLS; a cancel that arrives after the debit executed reports ALREADY_EXECUTED.